

202401465_Lab_4_Report

CONTINUATION AFTER STAGE 5 :

Prompts used :

Prompt Stage 6 : (User Stories Generation after Requirements are generated)

```
prompt_stage_6 = PromptTemplate(
    input_variables=["requirements"],
    template="""
    Based on the generated Functional Requirements (FRs), create Agile User Stories.
    Format: 'As a <stakeholder>, I want <goal>, so that <benefit>.'
    Include the Front of the card and the Back of the card (Acceptance Criteria) for each
    story.

    Requirements: {requirements}
    """
)
user_stories = run_stage_with_feedback(prompt_stage_6, {"requirements": requirements}, "Step 6: User Story Generation")
```

Prompt Stage 7 : (Invest Evaluation)

```
prompt_stage_7 = PromptTemplate(
    input_variables=["user_stories"],
    template="""
    You are an Agile Coach. Evaluate the following User Stories against the INVEST criteria:
    (Independent, Negotiable, Valuable, Estimable, Small, Testable).

    Flag any stories that fail and briefly explain why. If they pass, state that they meet the INVEST criteria.

    User Stories:
    {user_stories}
    """
)
invest_evaluation = run_stage_with_feedback(prompt_stage_7, {"user_stories": user_stories}, "Step 7: INVEST Criteria Check")
```

Prompt Stage 8 : (Sprint Grouping & Selection)

```
prompt_stage_8 = PromptTemplate(
    input_variables=["user_stories", "invest_evaluation"],
    template="""
    Based on the approved User Stories and their INVEST evaluation, organize the stories into 2-3 logical Sprints based on priority.
    Present the Sprints clearly. Then, ask the user which Sprint they would like to proceed with (the user will reply in the feedback prompt).
    """
)
```

```

        User Stories & Evaluation:
        {user_stories}
        {invest_evaluation}
        """
    )
    sprints = run_stage_with_feedback(prompt_stage_8, {"user_stories": user_stories, "invest_evaluation": invest_evaluation}, "Step 8: Sprint Grouping & Selection")

```

Prompt Stage 9 : (Prototype Generation)

```

prompt_stage_9 = PromptTemplate(
    input_variables=["sprints"],
    template="""
        Based on the Sprints identified above, and considering the user's feedback selection (if any),
        generate a working Python CLI prototype that implements the core functionality of the selected sprint's user stories.
        Output ONLY valid, raw Python code. Do not include markdown formatting or explanations.

        Sprints & Selection Context:
        {sprints}
        """
)
prototype_code_raw = run_stage_with_feedback(prompt_stage_9, {"sprints": sprints}, "Step 9: Prototype Generation")

```

Prompt Stage 10 : (Test Case Generation)

```

prompt_stage_10 = PromptTemplate(
    input_variables=["sprints", "prototype_code"],
    template="""
        Based on the selected sprint's Acceptance Criteria and the generated Python prototype, write a valid Python `unittest` script to test the prototype.
        The prototype is saved as a module named `prototype` (so you can `import prototype`).
        Output ONLY valid, raw Python code containing the unittests. Do not include markdown formatting.

        Sprint Context: {sprints}
        Prototype Code: {prototype_code}
        """
)
test_code_raw = run_stage_with_feedback(prompt_stage_10, {"sprints": sprints, "prototype_code": prototype_code}, "Step 10: Test Case Generation")

```

Prompt Stage 11 (Automated Testing Execution)

```

prompt_stage_11 = PromptTemplate(
    input_variables=["test_output"],
    template="""

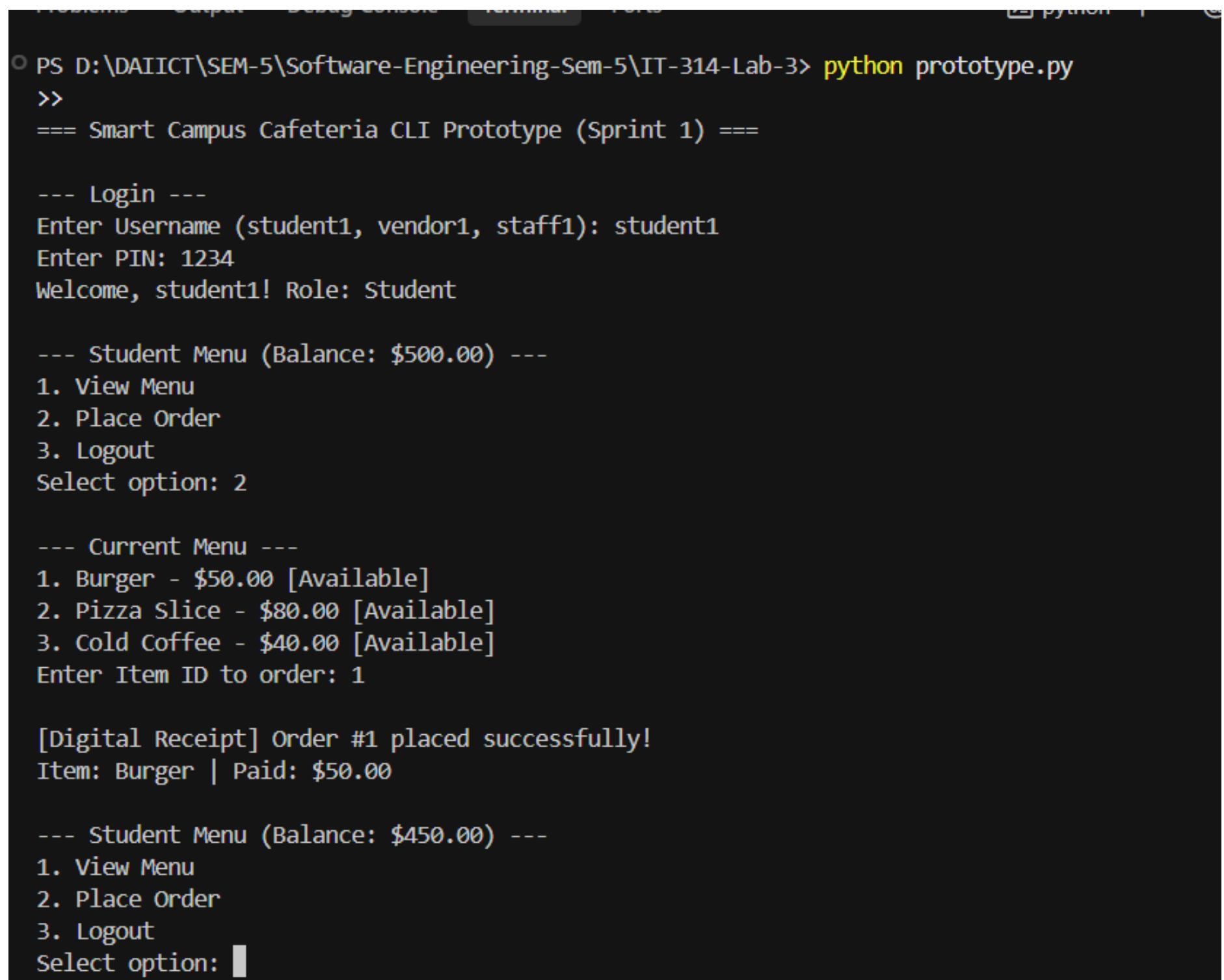
```

```
        The automated test suite has been executed against the prototype. Analyze the test execution output below.
        Report the final pass/fail results, and if any tests failed, briefly explain why.

        Test Execution Output:
        {test_output}
        """

    )
    test_report = run_stage_with_feedback(prompt_stage_11, {"test_output": test_output}, "Step 11: Automated Test Report")
```

CONSOLE Based Prototype Screenshots :



```
PS D:\DAIICT\SEM-5\Software-Engineering-Sem-5\IT-314-Lab-3> python prototype.py
>>
=== Smart Campus Cafeteria CLI Prototype (Sprint 1) ===

--- Login ---
Enter Username (student1, vendor1, staff1): student1
Enter PIN: 1234
Welcome, student1! Role: Student

--- Student Menu (Balance: $500.00) ---
1. View Menu
2. Place Order
3. Logout
Select option: 2

--- Current Menu ---
1. Burger - $50.00 [Available]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]
Enter Item ID to order: 1

[Digital Receipt] Order #1 placed successfully!
Item: Burger | Paid: $50.00

--- Student Menu (Balance: $450.00) ---
1. View Menu
2. Place Order
3. Logout
Select option: █
```

```

PS D:\DAIICT\SEM-5\Software-Engineering-Sem-5\IT-314-Lab-3> python prototype.py
>>
--- Student Menu (Balance: $450.00) ---
1. View Menu
2. Place Order
3. Logout
Select option: 1

--- Current Menu ---
1. Burger - $50.00 [Available]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]

--- Student Menu (Balance: $450.00) ---
1. View Menu
2. Place Order
3. Logout
Select option: █

```

Test Cases Output :

```

.
--- Login ---
Welcome, student1! Role: Student
.
--- Current Menu ---
1. Burger - $50.00 [Available]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]
Insufficient Campus ID balance.
.
--- Current Menu ---
1. Burger - $50.00 [Out of Stock]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]
Invalid item or item is out of stock.
.
--- Current Menu ---
1. Burger - $50.00 [Available]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]

[Digital Receipt] Order #1 placed successfully!
Item: Burger | Paid: $50.00
..
--- Current Menu ---
1. Burger - $50.00 [Available]
2. Pizza Slice - $80.00 [Available]
3. Cold Coffee - $40.00 [Available]
.
-----
Ran 9 tests in 0.005s

```

1. The Chosen Feedback Mechanism

For Lab 4, we implemented a **Hybrid Feedback System**:

- **Standard Human-in-the-Loop (HITL):** At the end of every pipeline stage, the terminal pauses, prints the AI's output, and waits for a human reviewer. The user can either press `Enter` to approve or type feedback (e.g., "Select Sprint 1") to steer the AI's next iteration.
- **LLM-as-a-Judge:** For Stage 7 (INVEST Criteria Check), the AI first acts as its own critic. It automatically grades the User Stories against the INVEST acronym, flags failures, and *then* hands the evaluation over to the human for final approval.

Reasoning: Why this specific mechanism?

- **Accountability & Control:** In Agile, blindly generating a prototype from unverified stories is disastrous. HITL guarantees a human Product Owner maintains absolute control over what is actually built in the Sprint.
- **Steering over Scrapping:** Instead of restarting the pipeline when a mistake happens, the feedback loop feeds the user's critique back into a "Refinement Prompt." This dynamically corrects the AI's path, saving API tokens and time.
- **Objective Grading:** Using "LLM-as-a-Judge" for the INVEST check provides a strict, objective filter before the human even has to read the stories, reducing the cognitive load on the reviewer.

Notable Prompting & Design Decisions

- **Context Chaining (The "Snowball" Effect):** Every prompt passes its generated output down to the next prompt. For example, Stage 8 (Sprint Selection) passes both the *approved* User Stories AND the *INVEST evaluation* to ensure the AI doesn't group a failing story into a sprint.
- **Controlling Prototype Scope:**
 - **Design Decision:** It is impossible for an LLM to reliably generate a full-scale web application in a single terminal prompt.
 - **Solution:** We scoped the prototype down to a single Python CLI script (`prototype.py`). The prompt explicitly forces the AI to only implement the exact stories selected by the user in the previous Sprint grouping stage.
- **Safe Automated Testing (Stage 11):**
 - **Design Decision:** We instructed the AI in Stage 10 to write tests using standard Python `unittest`.
 - **Execution Strategy:** Instead of just generating the code, the pipeline uses Python's native `subprocess.run()` to safely execute the tests in the background. It uses a `timeout=10` parameter to ensure the pipeline doesn't freeze if the AI hallucinates an infinite loop in the prototype code. It then reads the raw terminal output and has the LLM summarize it into a human-readable Test Report.